

Seeders y Factories

Nuestra aplicación necesita datos para funcionar, pero con cada migración desde cero estamos borrando esos datos lo cual nos impide testear adecuadamente, teniendo que ingresar datos a mano.

Para quitarnos este problema introduciremos el uso de Seeders y Factories.

Seeder

También conocido como “semillero”, un **seeder** es una clase que contiene la lógica para insertar datos en la base de datos. Se utilizan para sembrar la base de datos con información inicial, como usuarios administradores, configuraciones predeterminadas o datos de prueba.

Se crean en el directorio `database/seeder`s y se ejecutan desde la terminal.

Para que un seeder cree un registro podemos usar:

- Datos personalizados aleatorios
- **Faker**, una librería que permite definir datos no reales de forma aleatoria

No se recomienda usar datos reales que serán replicados en producción, dado que los seeders (y también los factories) se suben a nuestro repositorio de código y podrían exponer datos sensibles como nombres de usuarios, contraseñas y correos.

Factories

Las **factories** o fábricas son clases que se utilizan para generar modelos de Eloquent con datos falsos. Simplifican el proceso de creación de grandes cantidades de datos de prueba, como nombres, correos electrónicos, direcciones y mucho más.

Se crean en el directorio `database/factories` y están diseñadas para trabajar en conjunto con los seeders.

Acerca de Faker

Faker es una **biblioteca PHP** que genera datos falsos. Es la herramienta principal que Laravel usa en sus factories para crear datos de prueba, como nombres, direcciones, correos electrónicos, números de teléfono y otros de forma programática.

Como Laravel ya posee integrada esta librería, para usarla solamente es necesario usar el método **fake()** en conjunto con alguno de los métodos específicos de la librería.

Algunos ejemplos de datos que se pueden crear son los siguientes:

Datos Personales

- `fake()->name()`: Genera un nombre completo.

- `fake()->firstName()`: Genera un nombre de pila.
- `fake()->lastName()`: Genera un apellido.
- `fake()->email()`: Genera una dirección de correo electrónico válida y única.
- `fake()->password()`: Genera una contraseña aleatoria.

Datos de Dirección

- `fake()->address()`: Genera una dirección completa.
- `fake()->city()`: Genera un nombre de ciudad.
- `fake()->country()`: Genera un nombre de país.
- `fake()->postcode()`: Genera un código postal.

Datos de Negocios y Finanzas

- `fake()->company()`: Genera un nombre de empresa.
- `fake()->jobTitle()`: Genera un título de trabajo.
- `fake()->randomFloat(2, 10, 100)`: Genera un número decimal con 2 dígitos, entre 10 y 100.
- `fake()->randomNumber(5)`: Genera un número entero con 5 dígitos.

Datos de Fecha y Hora

- `fake()->date()`: Genera una fecha en formato 'YYYY-MM-DD'.
- `fake()->dateTime()`: Genera una fecha y hora.
- `fake()->date(Carbon::today())`: Genera una fecha en el pasado.
- `fake()->date(Carbon::now()->addDays(30))`: Genera una fecha en el futuro.

Datos de Texto

- `fake()->word()`: Genera una palabra aleatoria.
- `fake()->sentence()`: Genera una oración.
- `fake()->paragraph()`: Genera un párrafo.
- `fake()->text(200)`: Genera un texto de hasta 200 caracteres.

Opciones Misceláneas

- `fake()->boolean()`: Genera un valor booleano (true o false).
- `fake()->randomElement(['opcion1', 'opcion2', 'opcion3'])`: Elige un elemento aleatorio de un array.
- `fake()->imageUrl()`: Genera una URL de una imagen.
- `fake()->uuid()`: Genera un UUID.

Proceso de creación y ejecución

Vamos a crear seeders y factories para las dos tablas que tenemos en nuestro sistema: categorías y transacciones. Primero crearemos los seeders con los siguientes comandos:

```
php artisan make:seeder CategoriaSeeder
```

```
php artisan make:seeder TransaccionSeeder
```

La convención dictamina que el nombre de cada seeder deberá ser el del modelo asociado + la palabra Seeder al final.

Ahora creamos los dos factories:

```
php artisan make:factory CategoriaFactory
```

```
php artisan make:factory TransaccionFactory
```

Nuevamente, la convención dictamina que el nombre de cada factory deberá ser el del modelo asociado + la palabra Factory al final.

Definir los factories

Cada factory cuenta con un método llamado **definition()** que se encarga justamente de “definir” que datos se agregarán a un registro del modelo asociado. Editaremos estos métodos para agregar la lógica deseada.

CategoriaFactory:

```
public function definition(): array
{
    return [
        'nombre' => fake()->word(),
        'tipo' => fake()->randomElement(['gasto', 'ingreso']),
    ];
}
```

TransaccionFactory:

```
public function definition(): array
{
    return [
        'descripcion' => fake()->sentence(),
        'monto' => fake()->randomFloat(2, 1, 1000),
        'fecha_transaccion' => fake()->date(),
        'categoria_id' => Categoria::factory(),
    ];
}
```

```
    ];  
}
```

Definir los Seeders

Cada Seeder cuenta con un método llamado **run()** que es el encargado de definir como se poblará la tabla cuando se ejecuta ese Seeder particular. Editaremos esos métodos de ambos archivos.

CategoriaSeeder

```
public function run(): void  
{  
    // Creamos una categoría de gasto específica para pruebas  
    Categoria::create([  
        'nombre' => 'Alimentos',  
        'tipo' => 'gasto',  
    ]);  
    // Creamos una categoría de ingreso específica para pruebas  
    Categoria::create([  
        'nombre' => 'Salario',  
        'tipo' => 'ingreso',  
    ]);  
    // Creamos 10 categorías adicionales con la factory  
    Categoria::factory()->count(10)->create();  
}
```

Nota como las primeras dos categorías creadas en este Seeder no usan la definición de `CategoriaFactory`, sino que tienen datos específicos que nosotros deseamos que estén presente, lo cual es totalmente válido. Recuerda en este punto nunca poner datos sensibles que serán usados en producción, solamente datos de prueba. Luego creamos 10 categorías adicionales que si usan la `CategoriaFactory`, y por ende tendrán datos aleatorios.

TransaccionSeeder

```
public function run(): void  
{  
    Transaccion::factory()->count(100)->create();  
}
```

Dos cosas a destacar:

1. Aquí si usamos TransaccionFactory para definir 100 transacciones aleatorias.
2. En este caso no estamos vinculando la categoría de cada transacción porque en el archivo TransaccionFactory se crea una nueva categoría para cada transacción. Ten en cuenta que esto creará, por lo tanto, 100 categorías nuevas.

Para evitar el segundo punto, podríamos leer las categorías creadas, extraer sus ID de la tabla y luego asociarlos de forma aleatoria a cada Transaccion creada de la siguiente forma:

```
public function run(): void
{
    // Obtener todas las IDs de las categorías existentes
    $categoriaIds = Categoria::pluck('id');

    // Asociar cada una de las 100 transacciones con una categoría aleatoria
    Transaccion::factory()->count(100)->create([
        'categoria_id' => function () use ($categoriaIds) {
            return fake()->randomElement($categoriaIds);
        },
    ]);
}
```

Retocar los modelos

¿Cómo sabe Laravel que el archivo CategoriaFactory está vinculado al modelo Categoria? Esta tarea no se hace automáticamente, sino que debemos indicarlo en el modelo. En ambos modelos tendremos que agregar los siguientes cambios:

1. Importar el trait **HasFactory**

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
```

2. Indicar que el modelo usa dicho Trait.

```
class Categoria extends Model {
    use HasFactory;
    // Resto del modelo
}
```

Ejecutar los seeders

Una vez que tengamos todo configurado, tenemos al menos 3 formas de usar los seeders para poblar nuestras tablas:

1) Ejecutar un seeder particular

```
php artisan db:seed --class=CategoriaSeeder
```

Este comando permite indicar el nombre de la clase o archivo de un Seeder particular que deseamos ejecutar. Es la menos flexible, ya que si tenemos múltiples Seeders, ejecutarlos a todos será tedioso; pero es útil si deseamos poblar solamente una tabla para propósitos de testing.

2) Ejecutar todos los Seeders en conjunto

El comando db:seed visto anteriormente por defecto llama a un archivo que ya existe en la carpeta seeders: **DatabaseSeeder**. Este archivo define un único punto de entrada para poblar toda la base de datos. Por ello, podemos modificar su método **run()** para que llame automáticamente a los otros seeders de la siguiente forma:

```
$this->call([
    CategoriaSeeder::class,
    TransaccionSeeder::class,
]);
```

Ahora podemos ejecutar el comando sin argumentos, lo cual ejecutará el archivo DatabaseSeeder y, por ende, ambas tablas.

```
php artisan db:seed
```

3) Ejecutar una migración desde cero y los Seeders de forma conjunta

Esta es la variante más útil, dado que permite volver a recrear la estructura de nuestras tablas y adicionalmente agregar los datos a cada una de ellas.

El nuevo comando es el siguiente:

```
php artisan migrate:fresh --seed
```

Tips para trabajar con factories y seeders

- Nunca uses datos sensibles de producción en seeders. Recuerda que la idea de los seeders es poblar la BD con datos de prueba por motivos de testing. Usa faker u otra implementación que permita obtener datos aleatorios no reales.
- Nunca incluyas credenciales: contraseñas, claves de API, etc. Recuerda que los seeders por defecto se suben al control de versiones y esas credenciales quedarán registradas.
- Evita duplicados. Si estás usando datos que probablemente ya existan, haz una comprobación previa de los mismos en los seeders. Ejemplo: comprobar si un rol existe antes de crearlo:

```

public function run(): void
{
    // Se crea el rol solamente si no existe
    if (!Rol::where('name', 'admin')->exists()) {
        Rol::create(['name' => 'admin']);
    }
}

```

- Si tienes seeders que sean específicos de producción, sepáralos de los seeders de prueba o desarrollo.
- Mucho cuidado con ejecutar cualquier comando para poblar la BD en producción, recuerda que tu BD ya podría tener datos que serán sobrescritos, duplicados o simplemente agregarás datos de prueba. Siempre crea un backup previamente.

Cargar datos ya existentes

Si ya posees datos exportados de una base de datos en forma de comandos INSERT y deseas agregarlos a una tabla, puedes crear un Seeder (u opcionalmente, una migración o comando de consola en Laravel) que te permita leer el archivo y ejecutarlo. Para ellos crearemos un nuevo seeder:

```
php artisan make:seeder DatosInicialesSeeder
```

Luego moveremos el archivo SQL a una ubicación dentro de la carpeta database de nuestra aplicación (por ejemplo, database/sql) y modificaremos el seeder para leer el contenido del archivo y ejecutarlo.

```
use Illuminate\Database\Seeder;
```

```
use Illuminate\Support\Facades\DB;
```

```
class DatosInicialesSeeder extends Seeder
```

```

{
    public function run(): void
    {
        // Lee el contenido del archivo SQL
        $sql = file_get_contents(database_path('sql/datos.sql'));
        // Ejecuta el SQL
        DB::unprepared($sql);
    }
}

```

Por último, puedes ejecutar este nuevo seeder con cualquiera de las opciones vistas previamente, por ejemplo:

```
php artisan db:seed --class=DatosInicialesSeeder
```

Advertencia: el archivo **datos.sql** también puede contener datos que no deberían estar en el control de versiones. Por ello, se sugiere indicar a *git* que no rastree dicho archivo. La forma más fácil sería agregar la siguiente línea al archivo **.gitignore** que se encuentra en la raíz del proyecto:

```
/database/sql
```

De esta forma, todo lo que agreguemos a la carpeta `/database/sql` no será subido a git ni a Github, si fuera el caso.